

INSTALLING THE LIBRARIES

```
import os
import shutil
import cv2
import zipfile
import numpy as np
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D
from tensorflow.keras.preprocessing import image
```

READING THE DATASET & UNZIPPING THE DATASET

```
print(" Unzipping Dataset...")
zip_path = '/content/Dataset.zip'
extract_path = '/content/'

# Extract the uploaded zip file
with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Unzipping complete! You can see the 'Dataset' folder in your files.")
```

```
Unzipping Dataset...
Unzipping complete! You can see the 'Dataset' folder in your files.
```

SPLITTING THE DATA INTO TRAIN,TEST,VALIDATION

```
import os
import shutil
from sklearn.model_selection import train_test_split
from google.colab import files # Added to handle downloads

print("Splitting Data...")

TRAIN_PER_FOLDER = 24 # 24 x 4 folders = 96 Total Training Images
VAL_PER_FOLDER = 4 # 4 x 4 folders = 16 Total Validation Images

# PATH OF THE DIRECTORY
raw_data_dir = '/content/Dataset'
base_split_dir = '/content/Dataset_Split'

# Map original folders to target values
folder_to_value = {
    '1': '0.033',
    '2': '0.066',
    '3': '0.167',
    '4': '0.033'
}

# Create output directories
splits = ['train', 'val', 'test']
for split in splits:
    for value in set(folder_to_value.values()):
        os.makedirs(os.path.join(base_split_dir, split, value), exist_ok=True)

# Perform Stratified Split
for folder_name, target_value in folder_to_value.items():
    folder_path = os.path.join(raw_data_dir, folder_name)
    if not os.path.exists(folder_path):
        print(f"Warning: Folder {folder_name} not found!")
        continue

    images = [f for f in os.listdir(folder_path) if f.lower().endswith(('.png', '.jpg', '.jpeg', '.tif'))]

    # Split using your custom numbers from the Configuration section above
    train_imgs, remaining = train_test_split(images, train_size=TRAIN_PER_FOLDER, random_state=42)
    # Split the remaining images based on the VAL_PER_FOLDER value
    val_imgs, test_imgs = train_test_split(remaining, train_size=VAL_PER_FOLDER, random_state=42)
```

```

def copy_files(img_list, split_name):
    for img in img_list:
        src = os.path.join(folder_path, img)
        new_name = f"folder{folder_name}_{img}"
        dst = os.path.join(base_split_dir, split_name, target_value, new_name)
        shutil.copy(src, dst)

    copy_files(train_imgs, 'train')
    copy_files(val_imgs, 'val')
    copy_files(test_imgs, 'test')

print("Data splitting complete! Folders generated.")

# DOWNLOADING THE TEST IMAGE DATASET FOR TESTING THE MODEL
print("\nZipping the Test dataset for download...")
test_dir_path = os.path.join(base_split_dir, 'test')
zip_path = '/content/Unseen_Test_Images'

# Create a zip file of the test directory
shutil.make_archive(zip_path, 'zip', test_dir_path)

print("Downloading Unseen_Test_Images.zip to your computer...")
files.download(f"{zip_path}.zip")

```

Splitting Data...
Data splitting complete! Folders generated.

Zipping the Test dataset for download...
Downloading Unseen_Test_Images.zip to your computer...

```

# APPLYING SOME PREPROCESSING TECHNIQUES LIKE RESIZING & ROTATING ON DATA

print("Resizing and Rotating...")

IMG_SIZE = 224 # Converting images to 224*224 because the Resnet 50 model required this

def augment_folder(split_name):
    split_path = os.path.join(base_split_dir, split_name)
    processed_count = 0

    for class_folder in os.listdir(split_path):
        class_path = os.path.join(split_path, class_folder)
        if not os.path.isdir(class_path): continue

        images = os.listdir(class_path)
        for img_name in images:
            img_path = os.path.join(class_path, img_name)
            img = cv2.imread(img_path)
            if img is None: continue

            img_resized = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
            filename, ext = os.path.splitext(img_name)

            # The 4 distinct rotations
            rotations = {
                '0': img_resized,
                '90': cv2.rotate(img_resized, cv2.ROTATE_90_CLOCKWISE),
                '180': cv2.rotate(img_resized, cv2.ROTATE_180),
                '270': cv2.rotate(img_resized, cv2.ROTATE_90_COUNTERCLOCKWISE)
            }

            for angle, rotated_img in rotations.items():
                new_filename = f"{filename}_rot{angle}{ext}"
                cv2.imwrite(os.path.join(class_path, new_filename), rotated_img)

            os.remove(img_path) # Delete original to avoid duplicates
            processed_count += 1

    return processed_count * 4

# CREATING THE AUGMENTATION FOLDERS THAT INCLUDES THE AUGMENTED IMAGES
train_aug = augment_folder('train')
val_aug = augment_folder('val')
test_aug = augment_folder('test')
print(f"Augmentation complete! Total Images: Train({train_aug}), Val({val_aug}), Test({test_aug})")

```

Resizing and Rotating...
Augmentation complete! Total Images: Train(384), Val(64), Test(64)

```

# LOADING THE DATASET INTO THE MODEL & TRAINING THE MODEL USING RESNET 50 PRETRAINED MODEL

```

```

import tensorflow as tf
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, GlobalAveragePooling2D, Dropout, RandomFlip, RandomRotation, RandomZoom, RandomBr

print("Loading Data and Training REGRESSION Model...")
BATCH_SIZE = 16 # Batch size is 16 as at each training cycle 16 images will train
IMG_SIZE = 224

# The Scaling Trick: Map categorical labels to scaled math targets (x1000)
def map_to_floats(image, label):
    label = tf.cast(label, tf.int32)
    mapping = tf.constant([33.0, 66.7, 167.0], dtype=tf.float32)
    new_label = tf.gather(mapping, label)
    return image, new_label

# Load data strictly as 'int' so our mapping function works
# For training
train_dataset = tf.keras.utils.image_dataset_from_directory(
    '/content/Dataset_Split/train', image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE, label_mode='int'
).map(map_to_floats)

# For validation
val_dataset = tf.keras.utils.image_dataset_from_directory(
    '/content/Dataset_Split/val', image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE, label_mode='int'
).map(map_to_floats)

# For testing
test_dataset = tf.keras.utils.image_dataset_from_directory(
    '/content/Dataset_Split/test', image_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE, label_mode='int', shuffle=False
).map(map_to_floats)

# Extreme Online Data Augmentation
data_augmentation = Sequential([
    RandomFlip("horizontal_and_vertical"),
    RandomRotation(0.2),
    RandomZoom(0.1),
    RandomBrightness(0.2),
    RandomContrast(0.2)
], name="online_augmentation")

# Build ResNet50 Base & Deep Fine-Tuning
base_model = ResNet50(weights='imagenet', include_top=False, input_shape=(IMG_SIZE, IMG_SIZE, 3))
base_model.trainable = True

# Unfreeze the last 50 layers for deep texture learning
for layer in base_model.layers[:-50]:
    layer.trainable = False

# The Math CALCULATIONS OF THE MODEL
def calculate_expected_value(probabilities):
    target_values = tf.constant([33.0, 66.7, 167.0], dtype=tf.float32)
    expected_value = tf.reduce_sum(probabilities * target_values, axis=-1, keepdims=True)
    return expected_value

# Assemble the Model
model = Sequential([
    tf.keras.layers.Lambda(tf.keras.applications.resnet.preprocess_input, name='preprocessing'),
    data_augmentation,
    base_model,
    GlobalAveragePooling2D(),
    Dense(256, activation='relu'),
    Dropout(0.5),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(3, activation='softmax'), # CLASSIFICATION LAYER
    Lambda(calculate_expected_value) # REGRESSION CONVERSION LAYER
])

# Compile with Huber Loss and MAE Tracking
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4),
    loss=tf.keras.losses.Huber(),
    metrics=['mae']
)

early_stopping = tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=15, restore_best_weights=True)
reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)

# Start Training

```

```

print("\n Starting Training ... ")
history = model.fit(
    train_dataset,
    validation_data=val_dataset,
    epochs=16, # Epochs can be change as per the models requirements
    callbacks=[early_stopping, reduce_lr]
)

# Evaluate Error
print("\n Testing on Unseen Data ")
test_loss, test_mae = model.evaluate(test_dataset)
print(f"Final Test Mean Absolute Error : {test_mae:.2f}")

```

Loading Data and Training REGRESSION Model...

Found 384 files belonging to 3 classes.

Found 64 files belonging to 3 classes.

Found 64 files belonging to 3 classes.

Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/resnet/resnet50_weights_tf_dim_ordering_tf_kernels_0s_0us/step

Starting Training ...

Epoch 1/16

24/24 ————— 28s 246ms/step - loss: 42.3638 - mae: 42.8608 - val_loss: 41.9769 - val_mae: 42.4769 - learning_rate: 0.001

Epoch 2/16

24/24 ————— 3s 136ms/step - loss: 37.4196 - mae: 37.9082 - val_loss: 42.7750 - val_mae: 43.2521 - learning_rate: 0.0005

Epoch 3/16

24/24 ————— 5s 137ms/step - loss: 30.7049 - mae: 31.1571 - val_loss: 42.1026 - val_mae: 42.5354 - learning_rate: 0.00025

Epoch 4/16

24/24 ————— 3s 135ms/step - loss: 26.6180 - mae: 27.0420 - val_loss: 42.9387 - val_mae: 43.3792 - learning_rate: 0.000125

Epoch 5/16

24/24 ————— 3s 137ms/step - loss: 27.0793 - mae: 27.4739 - val_loss: 51.7673 - val_mae: 52.2276 - learning_rate: 6.25e-05

Epoch 6/16

24/24 ————— 4s 147ms/step - loss: 21.2994 - mae: 21.6420 - val_loss: 48.1400 - val_mae: 48.5022 - learning_rate: 3.125e-05

Epoch 7/16

24/24 ————— 3s 139ms/step - loss: 22.4059 - mae: 22.7462 - val_loss: 50.0599 - val_mae: 50.4215 - learning_rate: 1.5625e-05

Epoch 8/16

24/24 ————— 3s 139ms/step - loss: 22.9770 - mae: 23.3140 - val_loss: 50.6118 - val_mae: 50.9761 - learning_rate: 7.8125e-06

Epoch 9/16

24/24 ————— 3s 142ms/step - loss: 18.5992 - mae: 18.9187 - val_loss: 50.5891 - val_mae: 50.9612 - learning_rate: 3.90625e-06

Epoch 10/16

24/24 ————— 5s 139ms/step - loss: 21.3640 - mae: 21.6703 - val_loss: 47.6418 - val_mae: 48.0172 - learning_rate: 1.95312e-06

Epoch 11/16

24/24 ————— 3s 141ms/step - loss: 18.3860 - mae: 18.6734 - val_loss: 49.3977 - val_mae: 49.7649 - learning_rate: 9.76562e-07

Epoch 12/16

24/24 ————— 4s 156ms/step - loss: 19.7965 - mae: 20.0781 - val_loss: 48.3033 - val_mae: 48.6706 - learning_rate: 4.88281e-07

Epoch 13/16

24/24 ————— 4s 148ms/step - loss: 19.9141 - mae: 20.2067 - val_loss: 48.6715 - val_mae: 49.0502 - learning_rate: 2.44141e-07

Epoch 14/16

24/24 ————— 3s 143ms/step - loss: 18.7422 - mae: 19.0186 - val_loss: 49.3627 - val_mae: 49.7341 - learning_rate: 1.2207e-07

Epoch 15/16

24/24 ————— 3s 143ms/step - loss: 18.6776 - mae: 18.9530 - val_loss: 49.4428 - val_mae: 49.8323 - learning_rate: 6.10353e-08

Epoch 16/16

24/24 ————— 4s 151ms/step - loss: 16.4608 - mae: 16.7350 - val_loss: 50.0273 - val_mae: 50.4103 - learning_rate: 3.05176e-08

Testing on Unseen Data

4/4 ————— 0s 75ms/step - loss: 41.0230 - mae: 41.5230

Final Test Mean Absolute Error : 41.52

FINAL PREDICTION LAYER / OUTPUT LAYER

```

import numpy as np
import matplotlib.pyplot as plt
import io
from PIL import Image
from tensorflow.keras.preprocessing import image
from google.colab import files

# Uploading the unseen test image from the system
print("Click the button below to upload an image from your downloaded Test folder:")
uploaded = files.upload()

for filename in uploaded.keys():
    # Load and format the uploaded image
    img = image.load_img(io.BytesIO(uploaded[filename]), target_size=(224, 224))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0) # Create a batch of 1

    # Make the Hybrid Regression Prediction
    raw_prediction = model.predict(img_array, verbose=0)[0][0]

    # Correct Math: Reverse the Target Scaling Trick (/ 1000)
    final_roughness = raw_prediction / 1000.0

    # Display the image
    display_img = Image.open(io.BytesIO(uploaded[filename]))

```

```
plt.imshow(display_img)
plt.axis('off')
plt.show()

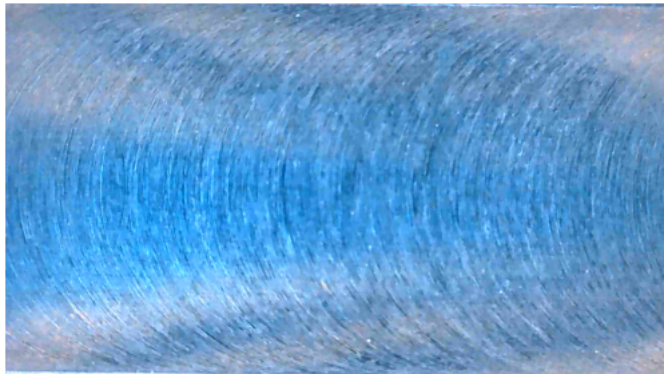
# Print the fluid, continuous prediction
print(f"\n Prediction Results for '{filename}' ")
print(f"Predicted Surface Roughness value is: {final_roughness:.3f}")
```

Click the button below to upload an image from your downloaded Test folder:

[Choose Files](#) folder2_2.4.jpg

folder2_2.4.jpg(image/jpeg) - 176442 bytes, last modified: 5/5/2026 - 100% done

Saving folder2_2.4.jpg to folder2_2.4 (1).jpg



Prediction Results for 'folder2_2.4 (1).jpg'
Predicted Surface Roughness value is: 0.063

```
# Save using the new native Keras format to avoid "config" errors
model.save('my_surface_model.keras')
print("Model saved as 'my_surface_model.keras'. Check your files panel!")
```

Model saved as 'my_surface_model.keras'. Check your files panel!

```
# FOR DEVELOPING THE UI FOR THE USERS WHERE THE USER CAN UPLOAD AN IMAGE & GETS SURFACE ROUGHNESS PREDICTION VALUE
!pip install gradio
```

```
Requirement already satisfied: gradio in /usr/local/lib/python3.12/dist-packages (5.50.0)
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (24.1.0)
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.13.0)
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.2.0)
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.136.1)
Requirement already satisfied: ffmpeg in /usr/local/lib/python3.12/dist-packages (from gradio) (1.0.0)
Requirement already satisfied: gradio-client==1.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.14.0)
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.2)
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.28.1)
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/local/lib/python3.12/dist-packages (from gradio) (1.11.6)
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.1.6)
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.0.3)
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.0.2)
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (3.11.8)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from gradio) (26.1)
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.2.2)
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (11.3.0)
Requirement already satisfied: pydantic<=2.12.3,>=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.12.3)
Requirement already satisfied: pydub in /usr/local/lib/python3.12/dist-packages (from gradio) (0.25.1)
Requirement already satisfied: python-multipart>=0.0.18 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.0.26)
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (6.0.3)
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.15.11)
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.1.7)
Requirement already satisfied: semantic-version~=2.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (2.10.0)
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.52.1)
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.13.3)
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.24.2)
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (4.15.0)
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/python3.12/dist-packages (from gradio) (0.46.0)
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.14.0->gradio) (2025.10.1)
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/lib/python3.12/dist-packages (from gradio-client==1.14.0->gradio) (13.1)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5.0,>=3.0->gradio) (3.13)
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from fastapi<1.0,>=0.115.2->gradio) (0.10.0)
Requirement already satisfied: annotated-doc>=0.0.2 in /usr/local/lib/python3.12/dist-packages (from fastapi<1.0,>=0.115.2->gradio) (0.0.2)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1.0,>=0.24.1->gradio) (2026.4.1)
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1.0,>=0.24.1->gradio) (1.0.7)
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1.0,>=0.24.1->gradio) (0.14.0)
Requirement already satisfied: filelock>=3.10.0 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (3.16.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.4.3 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (1.2.1)
Requirement already satisfied: tqdm==4.42.1 in /usr/local/lib/python3.12/dist-packages (from huggingface-hub<2.0,>=0.33.5->gradio) (4.42.1)
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2.9.0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=1.0->gradio) (2025.2)
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<=2.12.3,>=2.0->gradio) (0.7.0)
```

Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<=2.12.3,>=2.0.0)

Requirement already satisfied: click>=8.2.1 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (8.3.0)

Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (13.0.0)

Requirement already satisfied: rich>=12.3.0 in /usr/local/lib/python3.12/dist-packages (from typer<1.0,>=0.12->gradio) (13.9.0)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.8.2->pandas<3.0,py>=1.3.0)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer<1.0,>=0.12->gradio)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich>=12.3.0->typer<1.0,>=0.12->gradio)

Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich>=12.3.0->typer<1.0,>=0.12->gradio)

INSTALLING THE LIBRARIES WHICH ARE NEEDED FOR CREATING THE USER INTERFACE

```
import gradio as gr
import tensorflow as tf
import numpy as np
from PIL import Image

# LOGIC
def calculate_expected_value(probabilities):
    target_values = tf.constant([33.0, 66.7, 167.0], dtype=tf.float32)
    expected_value = tf.reduce_sum(probabilities * target_values, axis=-1, keepdims=True)
    return expected_value

# LOAD THE MODEL WITH CUSTOM OBJECTS
try:
    model = tf.keras.models.load_model(
        'my_surface_model.keras',
        custom_objects={
            'preprocess_input': tf.keras.applications.resnet.preprocess_input,
            'calculate_expected_value': calculate_expected_value
        },
        compile=False
    )
    print(" The Model Loaded Successfully!")
except Exception as e:
    print(f" Error: {e}")

def predict_roughness(input_img):
    # Standard ResNet Preprocessing
    img = Image.fromarray(input_img).resize((224, 224))
    img_array = np.array(img)
    img_array = np.expand_dims(img_array, axis=0)

    # Prediction
    raw_prediction = model.predict(img_array, verbose=0)[0][0]

    # Reverse the x1000 Scaling Trick
    final_value = raw_prediction / 1000.0

    # Formatting text with BOLD and 3 decimal places
    result_text = f"The **Surface Roughness value** of your image is : **{final_value:.3f}**"

    # Returning BOTH the text and the image for the bottom box
    return result_text, input_img

# UI DESIGN
with gr.Blocks(theme=gr.themes.Soft()) as demo:
    gr.Markdown("# AI SURFACE ROUGHNESS ANALYZER")

    # Amber Notice Box by using the HTML & INLINE CSS
    gr.HTML("""
<div style="background: rgba(255, 170, 0, 0.1); border-left: 5px solid #ffaa00; padding: 20px; border-radius: 8px; margin-bottom: 10px;">
<h4 style="color: #ffaa00; margin-top: 0;"> IMPORTANT GUIDELINES FOR SMOOTH PREDICTIONS</h4>
<ul style="color: #f8e5c4; font-size: 14px;">
<li>The material must be an <b>Aluminum sample</b>.</li>
<li>Captured under a <b>Digital Microscope</b>.</li>
<li>Captured using a <b>fixed lighting setup</b>.</li>
<li>Captured at a <b>fixed angle</b> and <b>fixed magnification</b>.</li>
</ul>
</div>
""")

    with gr.Row():
        with gr.Column():
            image_input = gr.Image(label="Choose Microscope Image")
            predict_btn = gr.Button("UPLOAD & ANALYZE IMAGE", variant="primary")

        with gr.Column():
            # Result Text Highlighted
            output_text = gr.Markdown(label="Analysis Result")
            # Result Image Preview (The missing part from your screenshot)
            output_image = gr.Image(label="Analyzed Prediction Result", height=300)
```

```
# Link button to function with two outputs
predict_btn.click(
    fn=predict_roughness,
    inputs=image_input,
    outputs=[output_text, output_image]
)

# LAUNCH THE WEBSITE
demo.launch(share=True)
```

The Model Loaded Successfully!
/tmp/ipykernel_584/2237012469.py:47: DeprecationWarning: The 'theme' parameter in the Blocks constructor will be removed in
with gr.Blocks(theme=gr.themes.Soft()) as demo:
Colab notebook detected. To show errors in colab notebook, set debug=True in launch()
* Running on public URL: <https://fa132872f9bfc58058.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU upgrades, run `gradio deploy` from the terminal in the

AI SURFACE ROUGHNESS ANALYZER

IMPORTANT GUIDELINES FOR SMOOTH PREDICTIONS

- The material must be an **Aluminum sample**.
- Captured under a **Digital Microscope**.
- Captured using a **fixed lighting setup**.
- Captured at a **fixed angle** and **fixed magnification**.

 Choose Microscope Image

 Analyzed Prediction Result



Drop Image Here

- or -

Click to Upload



Next steps:  [Deploy to Cloud Run](#)

```
# To delete all files and folders in the current directory
#!rm -rf
```